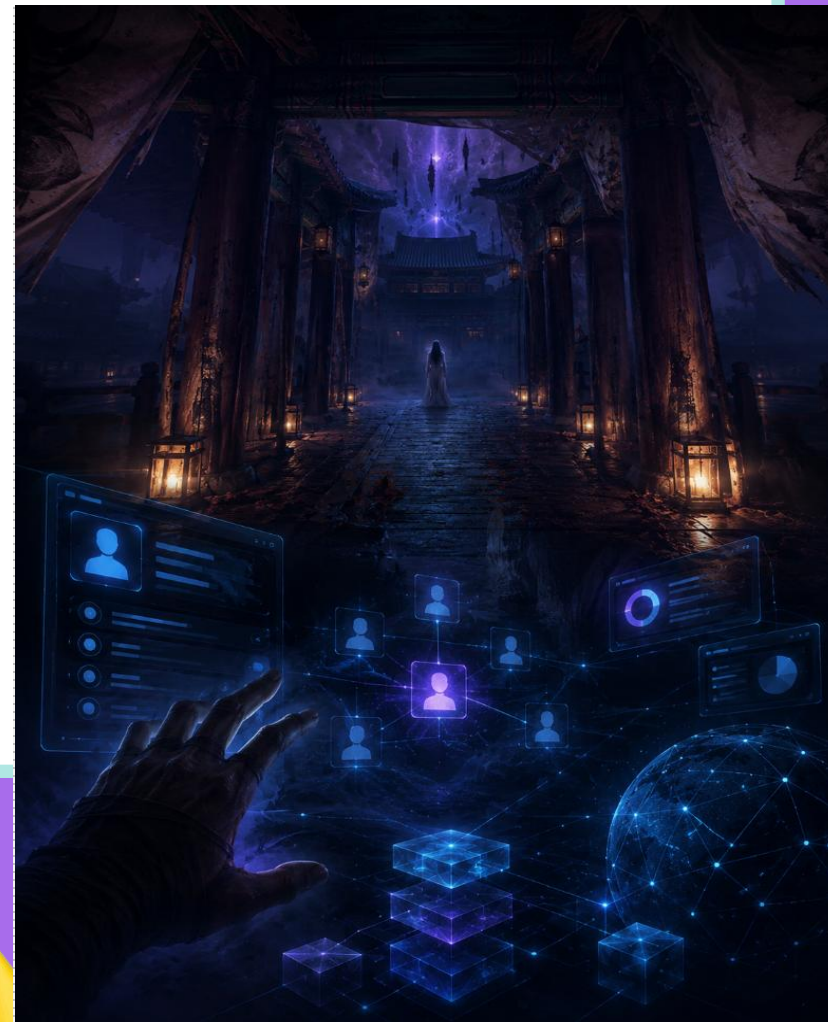


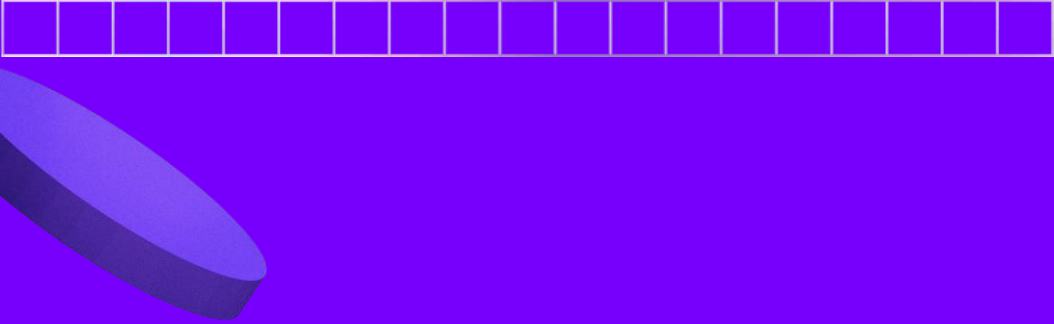


개발 포트폴리오

이재규

2026





목차

01. Warped Palace 프로젝트 소개

1인칭 공포게임 Warped Palace에
대한 간략한 소개 및 스크린샷

02. 설계 및 구현, 트러블 슈팅

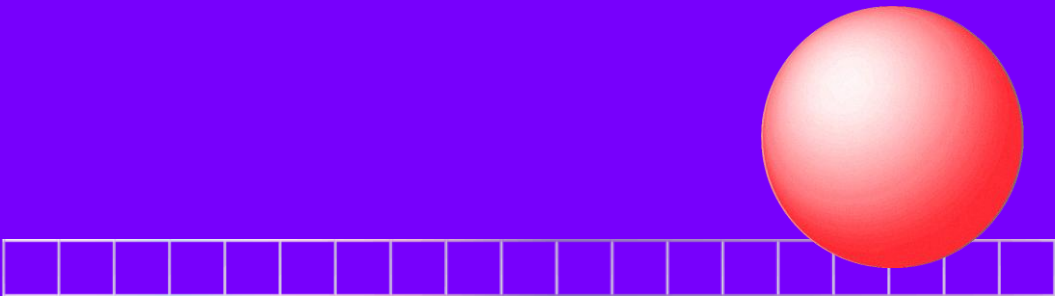
아키텍처 설계 및 구현사항, 구현
과정에서 생긴 문제사항 서술 및
개선사항

03. Muuiz 프로젝트 소개

멀티플레이 퀴즈게임 Muuiz에 대한
간략한 소개 및 진행 사진

04. 설계 및 구현, 트러블 슈팅

구현 및 설계과정에서의
문제 상황과 이에 대한 해결방안
적용 과정 및 결과



Warped Palace



Hide, Solve and Escape!

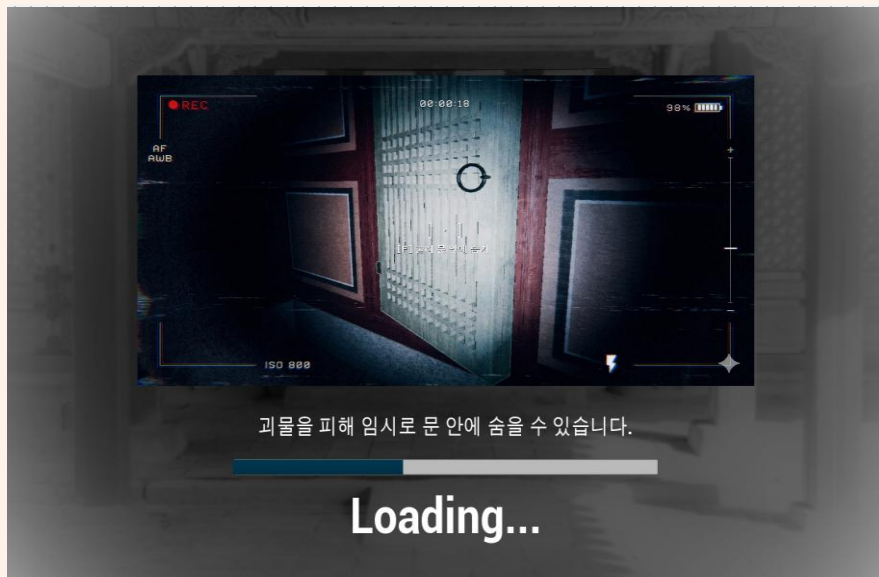
개발 인원 : 3인 개발

담당 파트: 아이템·인벤토리 시스템 및 인게임 오브젝트 상호작용
및 AI의 플레이어 아이템 사용 학습 구현

개발 기간 : 2026. 03 ~ 2026. 06

개발 스택 : Unreal, C++

깃허브 : https://github.com/lightaway01/Warped_Palace



Introduction

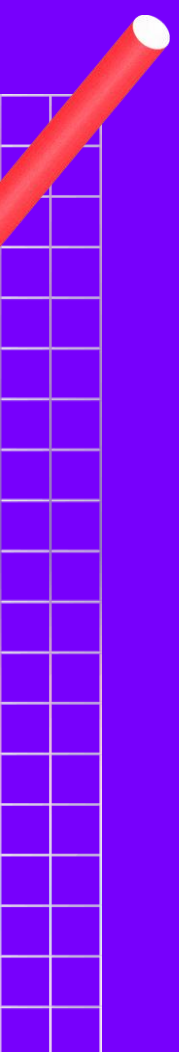
화성 행궁의 모습을 한 다른 세계에서 괴물인 장산범을 피해 퍼즐을 풀고 탈출하는 게임입니다.



Details

플레이어는 행궁속을 돌아다니며 아이템을 수집하고, 한국의 전통문화가 담긴 3개의 퍼즐을 풀어야 합니다.

괴물은 두 단계로 나뉜 AI와 플레이어의 아이템 사용을 학습하는 구조를 통해 실감나는 게임 플레이 경험을 제공합니다.



문제 상황 : 단일 배열의 역할 집중과 검색 비효율

- 기존 인벤토리는 하나의 배열이 아이템 보관, 검색, UI 표시 순서까지 모두 담당했습니다.
- 특정 아이템을 찾을 때마다 전체 배열을 순회해야 했습니다.
- 검색 성능을 위해 딕셔너리만 사용하면 UI 표시 순서를 안정적으로 관리하기 어려웠습니다.

해결 방법 접근

- 데이터베이스가 원본 데이터와 검색 인덱스를 분리해 관리하는 방식에서 착안하여, 인벤토리를 두 개의 자료구조로 구성했습니다.
 - TMap<ItemID, ItemList> -> 아이템 ID를 통한 빠른 조회
 - TArray<ItemData> -> 인벤토리 UI의 순서 유지
 - 아이템 추가삭제는 단일 메서드에서 두 자료구조를 함께 갱신하도록 구성했습니다.
- 

UI 표시용 배열과 아이템 검색용 인덱스 분리

```
UPROPERTY(VisibleAnywhere, Category = "Inventory | Contents") // UI 갱신용 아이템 배열
TArray<TObjectPtr<UItemBase>> InventoryContents;

UPROPERTY(VisibleAnywhere, Category = "Inventory | Indexing") // 아이템 ID 기반 검색용 Map
TMap<FName, FItemArrayWrapper> InventoryContentsMap;
```

```
struct FItemArrayWrapper
{
    GENERATED_BODY()

public:
    UPROPERTY(BlueprintReadWrite)
    TArray<TObjectPtr<UItemBase>> Items;
};
```

단일 변경 경로를 통한 데이터 정합성 유지

```
void UInventoryComponent::AddNewItem(UItemBase* Item, const int32 AmountToAdd)
{
    const int32 AddedIndex = InventoryContents.Add(*NewItem);
    InventoryContentsMap.FindOrAdd(NewItem->ID).Items.Add(*NewItem);

    InventoryTotalWeight += NewItem->GetItemStackWeight();

    OnInventoryItemAdded.Broadcast(
        NewItem,
        AddedIndex
    );
}
```

· 아이템 추가 로직을 AddNewItem()으로 단일화해 동일한 아이템 객체를 표시용 TArray와 검색용 TMap에 함께 등록했습니다.

두 자료구조의 갱신이 완료된 이후에만 이벤트를 전달하여 UI가 일관된 데이터를 참조하도록 했습니다.

자료구조 검증 결과(배열 + 해시 인덱스)

아이템 수	Array 조회	TMap 조회	Array & Map 삽입	Array & Map 삭제
20	1.46 ms	0.24 ms	0.27 μ s	0.46 μ s
100	2.69 ms	0.62 ms	1.04 μ s	2.65 μ s
1000	20.50 ms	0.78 ms	9.98 μ s	98.82 μ s

• Unreal Automation Test / UltemBase / 100,000 random lookups / median of 15 runs

결론

- TMap은 반복 ID 조회에서 안정적인 성능을 보였지만, 이중 구조는 삽입·삭제 시 두 자료구조 갱신과 정합성 관리 비용이 필요했습니다.
- 따라서 단순히 $O(1)$ 조회만 보고 선택하지 않고, 슬롯 순서 유지, ID 그룹 조회, 데이터 규모, 조회 빈도, 유지보수 비용을 함께 고려하였습니다.
- 현재 규모에서는 순수 검색 성능만으로 필수적인 구조는 아니었으나, UI 순서 유지와 ID 그룹 조회라는 서로 다른 요구사항을 분리할 수 있다는 점을 고려해 보조 인덱스 구조를 유지했습니다.

Muuiiz



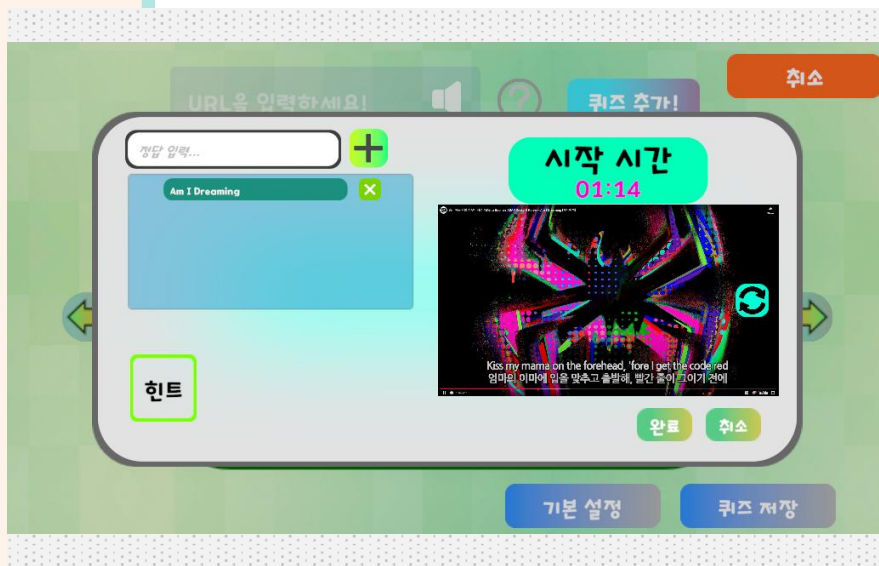
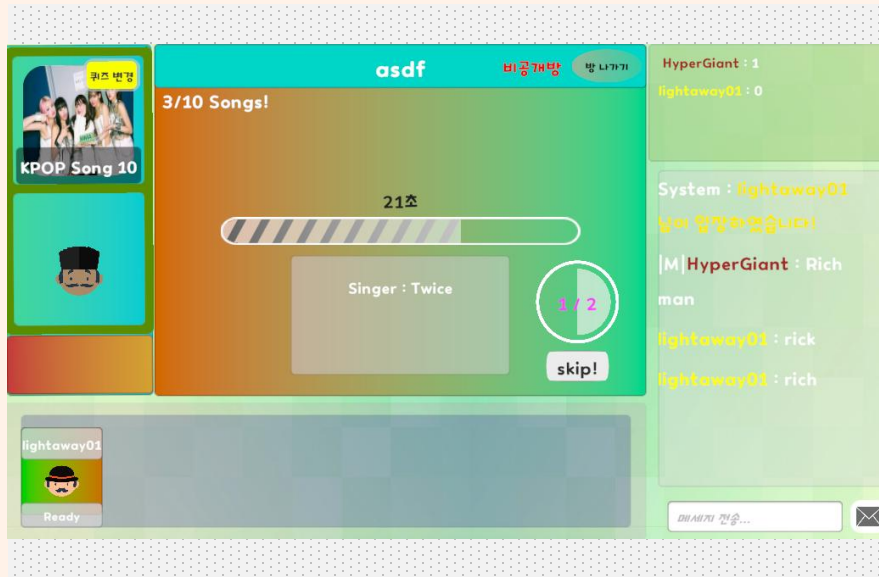
**Listen, Correct
and Enjoy!**

개발 인원 : 1인 개발

개발 기간 : 2025. 06 ~ 2025. 09

개발 스택 : Unity, Firebase(Database)

깃허브 : <https://github.com/lightaway01/Muuiiz>



Introduction

Muuiz 는 Music + Quiz의 줄임말로, 채팅을 통해 여러 사람들이 음악을 듣고 그 정답을 맞추으로써 점수를 얻기위해 경쟁하는 게임입니다.

Details

Unity Photon을 기반으로 다른 플레이어들과 실시간으로 채팅을 통해 퀴즈를 풀고,

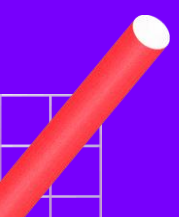
Unity Web View를 통해 Youtube URL 을 통하여 누구나 쉽고 간편하게 퀴즈를 제작할 수 있습니다.

멀티플레이 구현사항

- Unity와 Photon을 활용한 실시간 멀티플레이 음악 퀴즈 게임입니다.
MasterClient가 게임 진행 상태를 제어하고, 각 클라이언트가 Photon Custom Properties의 변경을 감지하여 퀴즈·영상·UI를 동기화하도록 구현했습니다.
- 이 과정에서 각각의 진행상태를 관리하기 위한 여러 개의 플래그 변수들이 활용되었습니다.

문제 상황 : 게임의 진행상태 관리

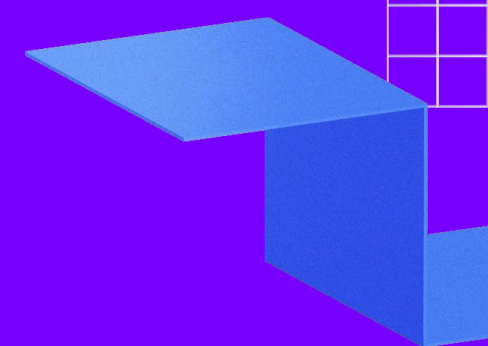
- 기능이 확장되면서 상태 값의 변경과 처리 로직이 여러 메서드에 분산되었습니다.
- (1) 높은 결합도: 하나의 상태 변경이 여러 관련 로직의 연쇄 수정으로 이어졌습니다.
- (2) 상태 추적 어려움: 여러 위치에서 속성 값을 변경해 현재 상태와 전이 원인을 추적하기 어려웠습니다.
- (3) 가독성·확장성 저하: 중첩 조건문이 누적되어 흐름 파악과 신규 상태 추가가 어려워졌습니다.



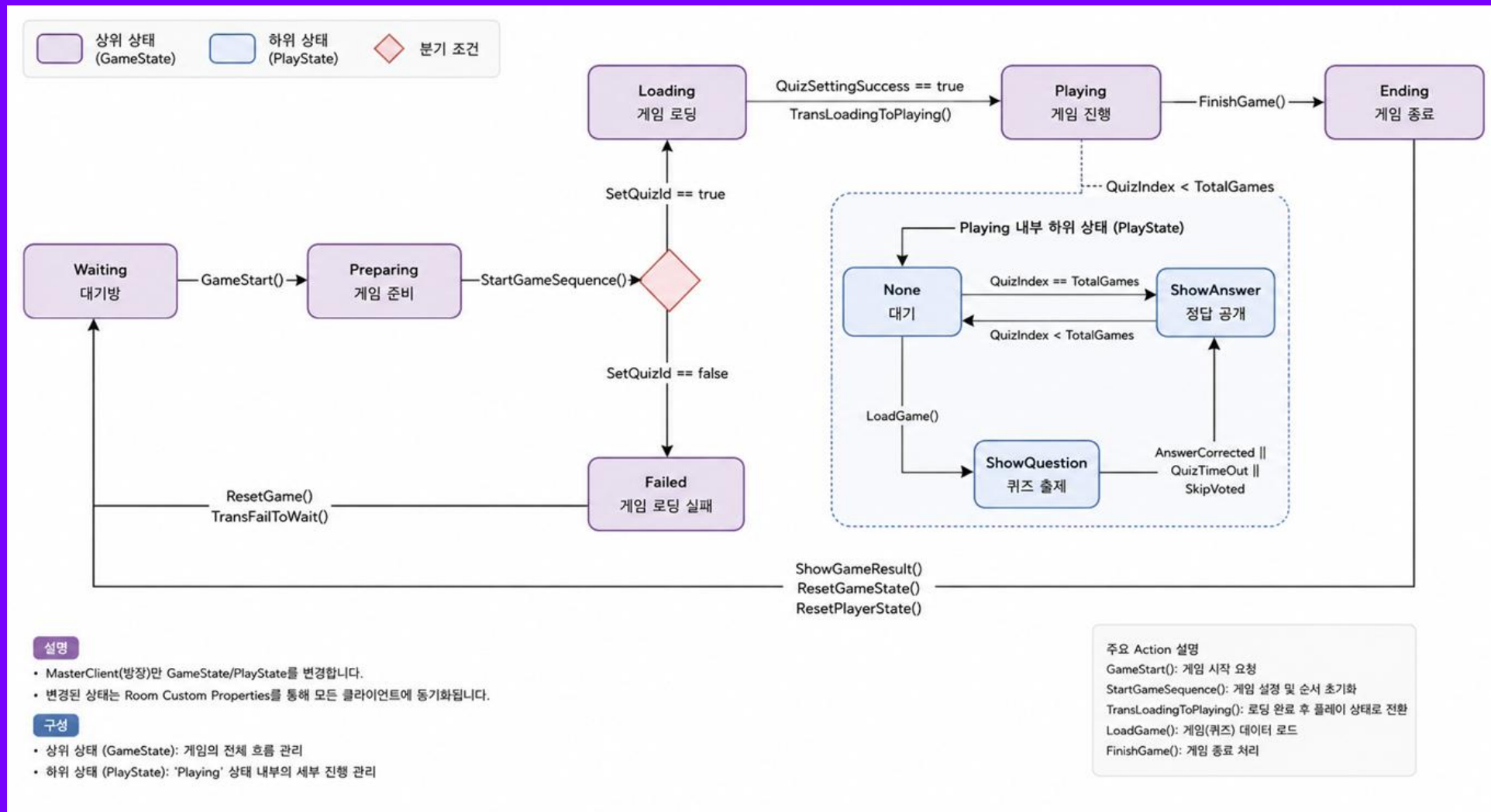
해결 방법 접근 : FSM/HFSM 기반 상태 관리 구조 도입

- 게임 흐름을 주요 상태로 구분하고, 각 상태에 대응하는 처리 로직을 분리했습니다.
- HFSM 적용: 복잡한 Playing 상태를 None, ShowQuestion, ShowAnswer의 하위 상태로 세분화했습니다.
- 결과: 상태 간 결합도와 변경 범위를 줄였으며, 신규 상태 추가와 버그 추적이 쉬워졌습니다.

FSM도입으로 인한 효과

- (1) 결합도 감소: 상태별 처리 로직이 구분되어 다른 흐름에 미치는 영향을 줄였습니다.
 - (2) 확장성 향상: 신규 상태와 전이 조건을 추가할 위치가 명확해졌습니다.
 - (3) 디버깅 효율화: 현재 상태와 전이 흐름을 기준으로 문제 발생 지점을 추적할 수 있게 되었습니다.
- 

FSM/HFSM 상태 전이 설계



Photon Custom Properties 기반 FSM 동기화 구현

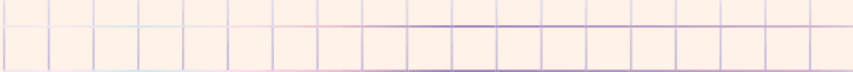
· 상위 GameState 처리

```
if (propertiesThatChanged.ContainsKey(Keyword.GAMESTATE))
{
    GameState state = (GameState)propertiesThatChanged[Keyword.GAMESTATE];
    switch (state)
    {
        case GameState.Waiting: // 대기방
            break;
        case GameState.Failed: // 게임 로딩 실패
            uiManager.GameStartFailed();
            TransFailtoWait();
            break;
        case GameState.Preaparing: // 게임 id 최종 확인 및 설정
            roundManager.StartGameSequence();
            break;
        case GameState.Loading:
            await roundManager.SetGameEnvrionment(); // 게임 기본 설정값 로딩
            break;
        case GameState.Playing:
            await roundManager.LoadGame(); //게임 로딩 및 퀴즈 풀이 시작
            break;
    }
}
```

· Playing 하위 PlayState 처리

Room Custom Property 변경 수신
→ GameState / PlayState 판별 → 상태별 로직 실행

```
switch (state)
{
    case PlayState.None:
        await roundManager.LoadGame(); // 다음 퀴즈 로딩
        break;
    case PlayState.ShowQuestion:
        roundManager.StartGameRoutine(); // 퀴즈 풀이 시작
        break;
    case PlayState.ShowAnswer:
        roundManager.EndRound(); // 정답 공개
        break;
    default:
        break;
}
```

THANKS!

읽어주셔서 감사합니다!

